DOUBLY LINKED LIST

What is a Doubly Linked List?

A Doubly Linked List is a linear data structure in which elements are stored in nodes, and each node contains **two pointers**:

- One pointer points to the **next node**
- Another pointer points to the **previous node**

Unlike singly linked lists, doubly linked lists allow traversal in **both forward and backward directions**.

Each node contains three parts:

1. **Previous field (Pointer)** – stores the address of the previous node
2. **Data field** – stores the actual value
3. **Next field (Pointer)** – stores the address of the next node

The address of the first node is stored in the **Head pointer**, and the last node's next pointer points to **NULL**. Similarly, the first node's previous pointer points to **NULL**.

Structure of a Node

A node in a doubly linked list consists of three components:

- **Prev Pointer** – reference to the previous node
- **Data** – the value stored in the node
- **Next Pointer** – reference to the next node

Representation:

```
[ Prev | Data | Next ]
```

Example Doubly Linked List:

```
NULL ← [10] ⇄ [20] ⇄ [30] → NULL
         ↑
        Head
```

Declaration of Doubly Linked List Node in C

The node structure is declared using a structure in C.

```
struct Node {
    int data;
    struct Node* prev;
```

```
    struct Node* next;
};
```

Explanation:

- `struct Node` defines the node type.
- `int data` stores the value of the node.
- `struct Node* prev` stores the address of the previous node.
- `struct Node* next` stores the address of the next node.

To create a pointer to the head node:

```
struct Node* head;
```

Here, `head` points to the first node of the doubly linked list.

Characteristics of Doubly Linked List

- Nodes are dynamically allocated in memory.
- Each node contains two pointers (prev and next).
- Traversal is possible in both directions.
- The list starts from the head node.
- The first node's prev pointer is NULL.
- The last node's next pointer is NULL.

Advantages of Doubly Linked List

1. Bidirectional Traversal

Traversal is possible both forward and backward.

2. Efficient Insertion and Deletion

Insertion and deletion are easier compared to singly linked lists because access to the previous node is available.

3. No Need to Traverse Backwards

No need to traverse from the head to find the previous node.

4. Flexible Data Structure

It can expand dynamically without requiring contiguous memory.

5. Useful for Complex Applications

Used where backward navigation is required.

Disadvantages of Doubly Linked List

1. Extra Memory Usage

Each node requires memory for two pointers (prev and next).

2. More Complex Implementation

Handling two pointers increases complexity.

3. More Operations Required

Updating both prev and next pointers during insertion/deletion.

4. Slower than Singly Linked List (Slightly)

Due to extra pointer handling.

Real-Life Use Cases of Doubly Linked List

Doubly linked lists are used in many real-world applications:

1. Navigation Systems

Allow moving forward and backward between locations.

2. Browser Navigation

Used for **forward and backward browsing** between web pages.

3. Music/Video Players

Users can move to next or previous media easily.

4. Undo/Redo Operations

Applications maintain actions where both undo and redo are possible.

5. Cache Implementation

Used in **LRU (Least Recently Used) Cache**.

6. Implementing Other Data Structures

Doubly linked lists are used to implement:

- Deques (Double-ended queues)

- Advanced data structures requiring bidirectional traversal

Key Points to Remember

- A doubly linked list consists of nodes with **three parts**: prev, data, and next.
- Each node points to both **previous and next nodes**.
- The first node's prev is NULL and the last node's next is NULL.
- Traversal is possible in **both directions**.
- It uses **more memory** than a singly linked list but provides greater flexibility.